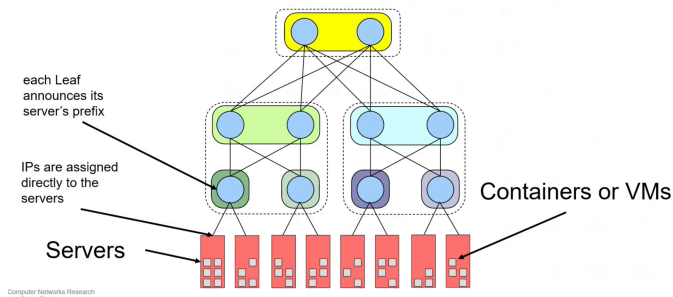


connecting the servers

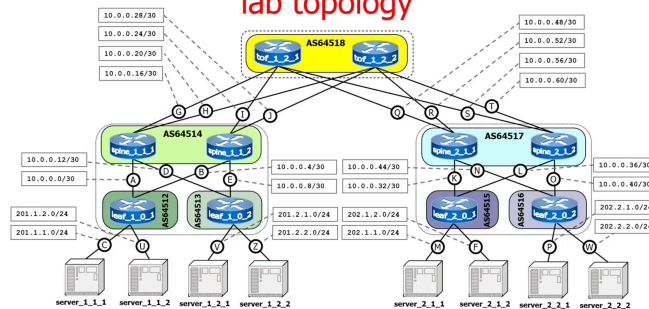


Proviamo ad implementare questa rete di un data center.

naming convention

- **tof_x_y_z**
 - x: plane number
 - y: level, always 2
 - z: ToF number
- **leaf_x_y_z**
 - x: PoD number
 - y: level, always 0
 - z: Leaf number
- **spine_x_y_z**
 - x: PoD number
 - y: level, always 1
 - z: Spine number
- **server_x_y_z**
 - x: PoD number
 - y: corresponding Leaf number
 - z: server number

lab topology



ToF configuration example

```

bgpd.conf
router bgp 64518
  timers bgp 3 9
  bgp router-id 192.168.0.14
  no bgp ebgp-requires-policy
  bgp bestpath as-path multipath-relax
  neighbor fabric peer-group
  neighbor fabric remote-as external
  neighbor fabric advertisement-interval 0
  neighbor fabric timers connect 10
  neighbor eth0 interface peer-group fabric
  neighbor eth1 interface peer-group fabric
  neighbor eth2 interface peer-group fabric
  neighbor eth3 interface peer-group fabric
  address-family ipv4 unicast
  neighbor fabric activate
  maximum-paths 64
  exit-address-family
        
```

- set keepalive and hold timers
- BGP multi-path relax
- create a peer-group
- all peers of the group are eBGP. The remote ASN is inferred
- set advertisement interval timer
- set connect timer
- add interfaces to peer-group for unnumbered peering
- enable A.F. to peer-group
- maximum Multi-Path possibilities

Configurazione della tof, top of fabric.

La cosa comoda è che fra i due router della tof devo cambiare solamente il bgp router id, il resto è tutto completamente identico.

Spine configuration example

bgpd.conf - part 1

```
router bgp 64514
timers bgp 3 9
bgp router-id 192.168.0.5
no bgp ebgp-requires-policy
bgp bestpath as-path multipath-relax

neighbor TOR peer-group
neighbor TOR remote-as external
neighbor TOR advertisement-interval 0
neighbor TOR timers connect 10
neighbor eth0 interface peer-group TOR
neighbor eth1 interface peer-group TOR

neighbor fabric peer-group
neighbor fabric remote-as external
neighbor fabric advertisement-interval 0
neighbor fabric timers connect 10
neighbor eth2 interface peer-group fabric
neighbor eth3 interface peer-group fabric
```

bgpd.conf - part 2

```
address-family ipv4 unicast
neighbor fabric activate
neighbor TOR activate
maximum-paths 64
exit-address-family
```

© Computer Networks Research

Leaf configuration example

bgpd.conf

```
router bgp 64512
timers bgp 3 9
bgp router-id 192.168.0.1
no bgp ebgp-requires-policy
bgp bestpath as-path multipath-relax

neighbor TOR peer-group
neighbor TOR remote-as external
neighbor TOR advertisement-interval 0
neighbor TOR timers connect 10
neighbor eth0 interface peer-group TOR
neighbor eth1 interface peer-group TOR

address-family ipv4 unicast
neighbor TOR activate
network 201.1.1.0/24
network 201.1.2.0/24
maximum-paths 64
exit-address-family
```

announce the server prefixes

Fine lezione del 15 dicembre
Inizio lezione del 16 dicembre

Facciamo un piccolo ripasso :

Multi path in un data center è importante perché ti consente di avere a disposizione diversi cammini, questo è buono per bilanciare il carico.

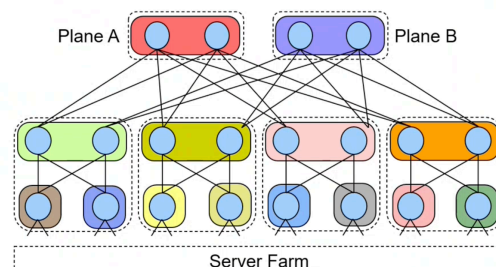
Nel nostro caso abbiamo visto equal cost multi path , che considera diversi cammini come simili se a costo uguale.

Se usi ecmp il forwarding viene fatto in modo tale che pacchetti della stessa connessione vengano instradati sullo stesso cammino , questo è facile da fare utilizzando tecniche hash.

Abbiamo visto che c'è traffico da nord a sud (da e per internet) e da ovest a est, il traffico del datacenter.

Ci sono diverse topologie ma noi ci concentriamo sui fat tree.

Nel datacenter il routing è bgp, ebgp nel particolare, costruito per motivi discussi in precedenza , assegnando un as in questo modo :



Con bgp bisogna fare un relax di ecmp.

In bgp con ecmp hai due cammini uguali se sono identici, per esigenza nei datacenter dobbiamo rilassare questa ipotesi affinché due cammini con la stessa lunghezza siano percepiti come identici.

```
Seleziona root@leaf_1_0_1: /

--- Startup Commands Log

++ ifconfig eth0 10.0.0.1/30 up
++ ifconfig eth1 10.0.0.5/30 up
++ ifconfig eth2 201.1.1.1/24 up
++ ifconfig eth3 201.1.2.1/24 up
++ /etc/init.d/frr start
Started watchfrr.

--- End Startup Commands Log

root@leaf_1_0_1:/#
```

```
root@leaf_1_0_1:/# route
Kernel IP routing table
Destination      Gateway         Genmask         Flags Metric Ref    Use Iface
0.0.0.0          0.0.0.0         0.0.0.0         u        0      0      0 eth0
10.0.0.4         0.0.0.0         0.0.0.0         u        0      0      0 eth1
201.1.1.0        0.0.0.0         0.0.0.0         u        0      0      0 eth2
201.1.2.0        0.0.0.0         0.0.0.0         u        0      0      0 eth3
201.2.1.0        10.0.0.2        0.0.0.0         u        20     0      0 eth0
201.2.2.0        10.0.0.2        0.0.0.0         u        20     0      0 eth0
202.1.1.0        10.0.0.2        0.0.0.0         u        20     0      0 eth0
202.1.2.0        10.0.0.2        0.0.0.0         u        20     0      0 eth0
202.2.1.0        10.0.0.2        0.0.0.0         u        20     0      0 eth0
202.2.2.0        10.0.0.2        0.0.0.0         u        20     0      0 eth0
root@leaf_1_0_1:/#
```

Con il comando route e basta non apprezziamo la differenza, mentre usando ip route:

```
root@leaf_1_0_1:/# ip route
10.0.0.0/30 dev eth0 proto kernel scope link src 10.0.0.1
10.0.0.4/30 dev eth1 proto kernel scope link src 10.0.0.5
201.1.1.0/24 dev eth2 proto kernel scope link src 201.1.1.1
201.1.2.0/24 dev eth3 proto kernel scope link src 201.1.2.1
201.2.1.0/24 nhid 12 proto bgp metric 20
    nexthop via 10.0.0.2 dev eth0 weight 1
    nexthop via 10.0.0.6 dev eth1 weight 1
201.2.2.0/24 nhid 12 proto bgp metric 20
    nexthop via 10.0.0.2 dev eth0 weight 1
    nexthop via 10.0.0.6 dev eth1 weight 1
202.1.1.0/24 nhid 12 proto bgp metric 20
    nexthop via 10.0.0.2 dev eth0 weight 1
    nexthop via 10.0.0.6 dev eth1 weight 1
202.1.2.0/24 nhid 12 proto bgp metric 20
    nexthop via 10.0.0.2 dev eth0 weight 1
    nexthop via 10.0.0.6 dev eth1 weight 1
202.2.1.0/24 nhid 12 proto bgp metric 20
    nexthop via 10.0.0.2 dev eth0 weight 1
    nexthop via 10.0.0.6 dev eth1 weight 1
202.2.2.0/24 nhid 12 proto bgp metric 20
    nexthop via 10.0.0.2 dev eth0 weight 1
    nexthop via 10.0.0.6 dev eth1 weight 1
root@leaf_1_0_1:/#
```

Vediamo che da un certo punto in poi hanno tutti due nexthop.

Aggiornamento del lab.conf :

```
# Multipath policy
# Docs here:
# https://www.kernel.org/doc/Documentation/networking/ip-
# sysctl.txt
# Possible values: 0 - Layer 3, 1 - Layer 4, 2 - Layer 3 or
# inner Layer 3 if present
# Default: 0
tof_1_2_2[sysctl]="net.ipv4.fib_multipath_hash_policy=1"
tof_1_2_1[sysctl]="net.ipv4.fib_multipath_hash_policy=1"
spine_1_1_1[sysctl]="net.ipv4.fib_multipath_hash_policy=1"
spine_1_1_2[sysctl]="net.ipv4.fib_multipath_hash_policy=1"
spine_2_1_1[sysctl]="net.ipv4.fib_multipath_hash_policy=1"
spine_2_1_2[sysctl]="net.ipv4.fib_multipath_hash_policy=1"
leaf_1_0_1[sysctl]="net.ipv4.fib_multipath_hash_policy=1"
```